CITS3002: Lab 3 - Solution

Exercise 1:

a) Write a Python program that asks the user to enter two binary numbers, each consisting of 8 bits long. The program must check each input to ensure it contains only the characters 0 and 1 and that its length is exactly 8 bits; if either input is invalid, the program should print an error message and terminate. If both inputs are valid, the program should compute the Hamming distance between the two numbers. Finally, the program should print the computed Hamming distance.

Answer: The Hamming distance can be computed by performing a bitwise XOR operation between the two binary numbers and then counting the number of '1's in the result, since each '1' indicates a position where the two bits differ.

b) Modify the program from Exercise 1 so that instead of receiving two inputs, it asks the user to enter four valid codewords, each consisting of 8 binary digits. The program must verify that each input contains only 0s and 1s and has a length of 8 bits; if any input is invalid, the program should print an error message and terminate. After receiving the four valid codewords, the program should compute and print the Hamming distance of the code formed by these codewords.

Answer: The Hamming distance of a code is defined as the minimum Hamming distance between any pair of distinct codewords in the code.

Exercise 2:

Write a Python program that receives an 8-bit binary data word from the user and generates the Hamming code for single-bit error correction. The program must first verify that the input contains exactly 8 bits and only the characters 0 and 1; otherwise, it should print an error message and terminate.

a) The program should output the complete Hamming codeword, which consists of the original 8 data bits and the 4 computed parity bits.

Answer: The program first receives an 8-bit binary input and verifies that it contains only 0s and 1s. The data bits are then placed in their appropriate positions within the 12-bit Hamming code structure, while the parity bits occupy the positions that are powers of two (1, 2, 4, and 8). Each parity bit is computed by performing an XOR operation on the corresponding data bits that it checks. Finally, the program combines the 8 data bits and the 4 computed parity bits to produce and print the complete 12-bit Hamming codeword.

b) By modifying the program from part a, write a Python program that receives a 12-bit Hamming codeword as input and detects whether an error has occurred in the code. The output of the program should simply indicate whether an error has occurred.

Answer: The program must first verify that the input consists of exactly 12 binary digits. It should then recover the original 8 data bits by removing the parity bits, and print the extracted data. Next, the program should recompute the Hamming code from the extracted data and compare the newly generated codeword with the received 12-bit input. If the two codewords are identical, the program should print that the input is correct; otherwise, it should report that the code contains an error.

Exercise 3:

a) Write a Python program that simulates the basic operation of a Stop-and-Wait communication protocol when the communication channel is free of noise. The program should include two functions: sender and receiver. The sender has four frames (A, B, C, and D) and sends one frame at a time to the receiver. When a frame is received, the receiver should display the received frame and send an acknowledgment (ACK) back to the sender. After the sender receives the ACK, it should proceed to send the next frame. The program should continue this process until all frames have been successfully delivered to the receiver and acknowledged.

Answer: The sender transmits one frame containing data to the receiver and then waits for an ACK. When the receiver gets the frame, it prints the received data and immediately sends an ACK back to the sender. This process continues until all frames are delivered successfully to the receiver. The sender and receiver operations are implemented as separate functions, and a loop controls the sequence of sending frames, receiving them, and acknowledging them.

b) Modify the program from Exercise 3-a to simulate a Stop-and-Wait protocol with transmission errors. Assume that the communication channel fails with a probability of 20%, meaning that either a transmitted frame or its ACK may be lost. For each frame, determine how many retransmissions are needed before it is successfully delivered to the destination.

Answer: The sender sends one frame at a time and waits for an acknowledgment (ACK) from the receiver before sending the next frame. A random function is used to simulate the loss of frames or ACKs in the communication channel. If either the frame or the ACK is lost, the sender does not receive the ACK and retransmits the same frame until the correct acknowledgment is received.

c) Repeat exercise 3-b with the channel failure probability increased to 50%, and compare the retransmission counts in the two cases for each frame.

Answer: Only the frame/ACK loss probability needs to be changed in the solution for part (b).